

LEUPHANA UNIVERSITY OF LÜNEBURG

Faculty of Management and Technology

Bachelor's Thesis

**Developing a Semantic Knowledge Graph of the Academic
Structure of Leuphana University**

Submitted by

Nadeem Hakimi

Matriculation Number: 4000409

Supervisor I: Dr. Debayan Banerjee

Supervisor II: Soham Majumder

March 2026

Declaration of Authorship

I hereby declare that this thesis is my own work and that I have not used any sources or aids other than those stated. All passages taken from publications or other sources have been marked as such.

Hamburg, 19.03.2026

[Student Signature]

Declaration of AI Tool Usage

In accordance with the academic integrity guidelines applicable to this thesis, I hereby disclose my use of artificial intelligence tools during the preparation of this work.

The following AI-assisted tools were used:

- OpenAI GPT-5.2 is integrated as a functional technical component of the Natural Language Query interface described in Chapter 7. This is a system use of AI, not merely an aid to writing.
- Code-completion tools (e.g., GitHub Copilot, Claude) were used to assist in developing Python scripts for web scraping and RDF generation.

All AI-generated content was critically reviewed, verified, and adapted by the author. The intellectual contribution, research design, evaluation methodology, and conclusions of this thesis are the author's own. AI tools were used as productivity aids only.

This declaration is made in accordance with applicable examination regulations, which require explicit disclosure of AI tool usage.

Hamburg, 19.03.2026

[Student Signature]

Abstract

This thesis presents the design and implementation of a semantic knowledge graph that captures the academic structure of Leuphana University of Lüneburg. The project addresses the problem of fragmented organisational knowledge distributed across the university's web presence by constructing a unified, queryable semantic database grounded in established Semantic Web standards.

The work proceeded through three principal stages. First, an OWL ontology was developed to formalise the core university entities--including schools, institutes, chairs, professors, and study programmes--and the relationships among them. Second, a Python-based web scraper was implemented to extract structured data from leuphana.de automatically. Third, the extracted data were transformed into RDF triples and imported into GraphDB, a dedicated semantic graph database.

The resulting knowledge graph comprises approximately 24,356 RDF triples representing 2,269 distinct entities, including 8 schools, 36 institutes, 23 chairs, over 1,000 persons, 76 study programmes, and 917 courses. The system successfully resolves the predefined competency questions--such as identifying professors by school, tracing organisational hierarchies, and listing available student assistant positions--with query response times consistently below 200 milliseconds.

In addition, an interactive D3.js visualisation application was developed, augmented by a natural language query interface that employs LangChain and OpenAI's GPT-5.2 to translate plain-English questions into valid SPARQL queries. The natural language interface achieves an average accuracy of approximately 85% across diverse query types. The project demonstrates that semantic web technologies can meaningfully improve the accessibility and interconnection of institutional information.

Keywords: Knowledge graph, Semantic web, RDF, OWL, Web scraping, GraphDB, University information system, Natural language querying, SPARQL.

Table of Contents

Declaration of Authorship	I
Declaration of AI Tool Usage	I
Abstract	II
Table of Contents	III-V
List of Abbreviations	VI
1. Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Project Goals	3
1.4 Research Questions	3
1.5 Methodological Framework	4
1.6 Thesis Structure	4
2. Background and Related Work	5
2.1 Knowledge Graphs	5
2.2 Semantic Web Technologies	5
2.2.1 RDF (Resource Description Framework)	5
2.2.2 OWL (Web Ontology Language)	6
2.2.3 SPARQL	7
2.2.4 SHACL (Shapes Constraint Language)	7
2.3 Related Work	8
2.3.1 Existing University Ontologies	8
2.3.2 Comparison of Existing Ontologies	9
2.3.3 University Knowledge Graph Projects	9
2.3.4 Web Information Extraction	10
2.4 Tools and Technologies	10
3. Requirements Analysis	11
3.1 Scope Definition	11
3.1.1 Entities Included	11
3.1.2 Entities Excluded	11
3.2 Competency Questions	11
3.3 Data Sources	12
3.4 Technical Requirements	12
4. Ontology Design	13
4.1 Design Methodology	13
4.2 Class Hierarchy	13
4.2.1 Organisational Classes	14
4.2.2 Person Classes	14
4.2.3 Academic Classes	14

4.3 Properties	14
4.3.1 Object Properties	14
4.3.2 Data Properties	15
4.4 External Vocabulary Reuse	15
4.5 Key Design Decisions	16
4.6 Validation	16
5. Web Data Extraction	17
5.1 Website Analysis	17
5.2 Implementation	17
5.2.1 Technology Stack	17
5.2.2 Architecture	17
5.2.3 Extraction Pipeline	17
5.2.4 Extraction Example	18
5.3 Challenges and Solutions	19
5.3.1 Inconsistent HTML	19
5.3.2 Missing Information	19
5.3.3 Rate Limiting	20
5.3.4 Ethical Considerations	20
6. Knowledge Graph Construction	21
6.1 RDF Generation	21
6.1.1 URI Design	21
6.1.2 Output Formats	21
6.2 GraphDB Import	21
6.2.1 Tool Selection Rationale	21
6.2.2 Repository Configuration	22
6.3 Data Integration	22
6.4 SHACL Validation	22
7. Natural Language Query Interface	23
7.1 Architecture	23
7.2 Prompt Engineering	23
7.3 Retry Mechanism	24
7.4 User Interface Integration	24
8. Evaluation Methodology	25
8.1.1 Triple-Level Evaluation	25
8.1.2 Evaluation Scope	26
8.2 Quantitative Results	26
8.2.1 Knowledge Graph Statistics	26
8.2.2 Entity Counts by Type	26
8.2.3 Competency Question Coverage	27
8.3 Ontology Evaluation	28
8.4 Extraction Evaluation	29
8.5 Query Performance	29
8.6 NLQ Accuracy Evaluation	30
8.7 Evaluation Against Project Criteria	30

8.8 Limitations	31
9. Demonstration	32
9.1 SPARQL Query Examples	32
9.1.1 List All Schools	32
9.1.2 Professors in the School of Sustainability	33
9.1.3 Organisational Hierarchy	33
9.2 Visualisation	34
9.3 Practical Use Cases	34
10. Discussion	35
10.1 Reflection on Research Questions	35
10.2 Contributions	35
10.3 Challenges Encountered	36
10.4 Future Work	36
11. Conclusion	37
References	38
Appendices	42
Appendix A: Project Deliverables	42
Appendix B: Repository Structure	42
Appendix C: Setup Instructions	42
Appendix D: SPARQL Queries	42

List of Abbreviations

Abbreviation	Meaning
CQ	Competency Question
DSR	Design Science Research
FOAF	Friend of a Friend (vocabulary)
KG	Knowledge Graph
LLM	Large Language Model
NLQ	Natural Language Query
OWL	Web Ontology Language
RDF	Resource Description Framework
RDFS	RDF Schema
SHACL	Shapes Constraint Language
SPARQL	SPARQL Protocol and RDF Query Language
URI	Uniform Resource Identifier

1. Introduction

1.1 Motivation

Universities maintain increasingly complex organisational structures encompassing multiple schools, dozens of institutes, hundreds of academic staff, and a wide range of study programmes. At Leuphana University of Lüneburg, this information is distributed across hundreds of individual web pages with no centralised, machine-readable representation. A professor may appear on an institute page, a research group page, and a school overview page, yet linking these references requires extensive manual effort. There is no cross-referencing mechanism that would allow a prospective student to answer questions such as "Which professors in the School of Sustainability supervise theses on my topic?" or "Which institutes belong to the School of Management and Technology?" without navigating dozens of pages.

Knowledge graphs offer a principled solution to this problem. By representing institutional information as a structured network of entities and relationships (**see Figure 1.1 for a conceptual example**), complex queries that would otherwise require manual aggregation can be answered programmatically and instantly. Knowledge graphs have demonstrated their value in comparable settings: Google's Knowledge Graph powers the information panels shown in search results [1], and similar systems have been deployed for product discovery, social networking, and enterprise search [2]. Applying these techniques to a university's organisational data promises to improve both the accessibility and the interconnection of institutional information.

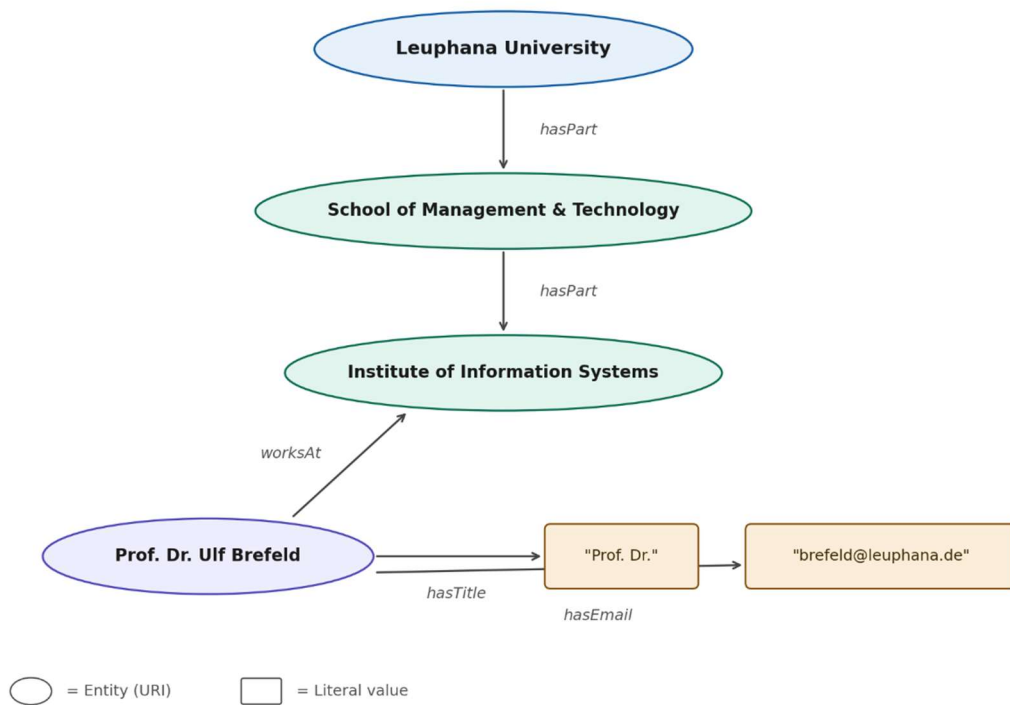


Figure 1.1: Conceptual example of a Knowledge Graph.

1.2 Problem Statement

Universities maintain extensive repositories of information regarding their organisational structure, personnel, and academic programmes. However, this information typically suffers from several interrelated limitations: (1) it is dispersed across numerous web pages of varying formats and designs; (2) it is authored for human consumption rather than machine processing, rendering automated analysis infeasible; (3) it lacks explicit semantic relationships between related entities, such that the connection between a professor and the research centre she directs must be inferred manually; and (4) it is difficult to query or aggregate programmatically, since the underlying data are not structured according to any formal schema.

The central problem addressed by this thesis is as follows: How can unstructured web content related to a university's academic structure be transformed into a structured, queryable knowledge representation that preserves entities and their interrelationships?

1.3 Project Goals

As outlined in the project description, this thesis aims to achieve six main objectives. First, it designs an ontology to capture the key entities at Leuphana—such as professors, departments, chairs, research groups, staff, courses, and student assistant positions—along with their relationships. Second, it extracts information from the public Leuphana website through automated web scraping. Third, it transforms the extracted data into RDF triples according to the ontology. Fourth, it builds a populated knowledge graph within an RDF store (GraphDB). Fifth, it evaluates the ontology, extraction quality, and knowledge graph quality. Sixth, it demonstrates the knowledge graph through SPARQL queries and a web-based visualization. Additionally, a natural language query interface was created using LangChain and GPT-5.2 to convert plain-English questions into SPARQL queries, improving accessibility for non-technical users. The complete source code for all project components is publicly available on GitHub.¹

1.4 Research Questions

Central to this project is the development of an ontology—a formal, machine-readable framework that defines the key concepts (such as people, institutions, and courses) and their relationships within a specific domain [11]. With this foundation, this thesis addresses three research questions:

RQ1: How well can ontology design be used as the representation of the academic structure of a German university, and at the same time be extended in the future?

RQ2: Which methods effectively extract structured information from semi-structured university web pages?

RQ3: What can be done to validate and present the resulting knowledge graph to the end users, including those who are not technically inclined?

¹ <https://github.com/Nadeemhakimi/Leuphana-knowledge-graph>

1.5 Methodological Framework

This project adopts a Design Science Research (DSR) methodology, as outlined by Hevner et al. [3] and Peffers et al. [4]. DSR focuses on creating and evaluating IT artifacts designed to address specific organizational problems. The methodology includes six activities: problem identification, objective setting, design and development, demonstration, evaluation, and communication. In this context, the IT artifacts include the ontology, the web scraper, the knowledge graph, and the natural language query interface. Each artifact was developed iteratively and assessed against predefined competency questions and quality metrics, consistent with the DSR evaluation framework.

1.6 Thesis Structure

The rest of this thesis is organized as follows. Chapter 2 covers background on knowledge graphs, semantic web technologies, and related work. Chapter 3 discusses the requirements analysis and scope definition. Chapter 4 explains the ontology design process. Chapter 5 describes the web scraping implementation. Chapter 6 explains RDF generation and database import. Chapter 7 details the natural language query interface. Chapter 8 shows the evaluation results. Chapter 9 demonstrates the system with example queries and visualizations. Chapter 10 explores the findings, limitations, and future research directions. Chapter 11 wraps up the thesis.

2. Background and Related Work

2.1 Knowledge Graphs

A knowledge graph is a structured representation of real-world entities and the relationships that hold among them [5]. Unlike traditional relational databases, which store data in tabular form, knowledge graphs represent information as a network of nodes (entities) connected by labelled edges (relationships). Ehrlinger and Woss [6] define a knowledge graph as a system that acquires and integrates information into an ontology and applies a reasoner to derive new knowledge. This definition emphasises the dual role of knowledge graphs as both repositories of factual assertions and platforms for inferential reasoning.

The concept gained mainstream prominence when Google announced its Knowledge Graph in 2012, which underpins the information panels displayed in search results [1]. Since then, knowledge graphs have been adopted by numerous major technology companies, including Microsoft with Satori (Bing), Amazon with its product graph, and Facebook with its social graph [2]. These industry-scale deployments have demonstrated the versatility of the knowledge graph paradigm for tasks ranging from question answering to recommendation systems.

Knowledge graphs offer several distinctive advantages for information management. Their schema is flexible and can evolve without requiring wholesale database restructuring--a property sometimes referred to as schema-on-read [7]. They provide a natural representation of complex, interconnected data that mirrors the relational structure of real-world domains. They support inference and reasoning over relationships through logical formalisms such as OWL and RDFS [8]. And they facilitate the integration of data from multiple heterogeneous sources, a capability that is particularly relevant to the present project, where information must be aggregated from dozens of differently structured web pages.

2.2 Semantic Web Technologies

2.2.1 RDF (Resource Description Framework)

Resource Description Framework (RDF) offers the basic data model of the knowledge graph in the paradigm of the Semantic Web [9]. RDF is a form of information expressed in triples

having the following structure: (subject, predicate, object) with each of the elements being defined by a globally unique Uniform Resource Identifier (URI). This triple-based model permits such an arbitrary number of complex structures of graphs out of simple atomic statements. RDF data can be serialised in many different formats, such as human-readable Turtle, syntactically simple and high-volume imports N-Triples, and integration with web applications JSON-LD [10].

2.2.2 OWL (Web Ontology Language)

OWL is used to define ontologies as formal, explicit specifications of the concepts and relationships that exist within a domain [11], [12]. Gruber’s seminal definition characterises an ontology as an explicit specification of a conceptualisation [11]. An ontology in OWL defines classes (categories of entities, such as Professor or Institute), properties (relationships and attributes), constraints (rules governing valid data), and class hierarchies (for example, JuniorProfessor is a subclass of Professor). The use of ontologies enables both semantic interoperability across systems and automated reasoning over the knowledge graph [13].

The following OWL fragment, expressed in Turtle syntax, illustrates how the class hierarchy for academic staff is defined in the present ontology:

```
@prefix leuph: <http://leuphana.de/ontology#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .

leuph:Professor a owl:Class ;
  rdfs:subClassOf leuph:AcademicStaff ;
  rdfs:label "Professor"@en ,
    "Professor"@de .

leuph:JuniorProfessor a owl:Class ;
  rdfs:subClassOf leuph:Professor ;
  rdfs:label "Junior Professor"@en ,
    "Juniorprofessor"@de .
```

Listing 2.1: OWL class hierarchy for academic staff in Turtle syntax

This declaration states that JuniorProfessor is a subclass of Professor, which in turn is a subclass of AcademicStaff. An OWL reasoner can therefore automatically infer that any individual classified as a Junior Professor is also a Professor and an Academic Staff member.

2.2.3 SPARQL

The standard query language of RDF data is called SPARQL and plays a similar role to SQL on relational databases [14]. SPARQL uses graph pattern matching to find triples that meet the required conditions, and it supports such operations as filtering, aggregation, optional matching, and federated queries across distributed endpoints. The flexibility of SPARQL 1.1 makes it suitable for both simple lookups and more complex analytical queries of knowledge graphs. For example, the following SPARQL query retrieves the names of all professors affiliated with the School of Sustainability, together with the institute at which each professor works:

```
SELECT ?name ?institute WHERE {  
  
  ?p a leuph:Professor .  
  
  ?p foaf:name ?name .  
  
  ?p leuph:worksAt ?i .  
  
  ?i leuph:partOf ?s .  
  
  ?s leuph:name "School of Sustainability" .  
  
  ?i leuph:name ?institute .  
  
}
```

This query matches all entities of type Professor, traverses the worksAt and partOf relationships to reach the school level, and filters by school name—illustrating SPARQL's ability to navigate multiple relationship hops within a single query.

2.2.4 SHACL (Shapes Constraint Language)

The Shapes Constraint Language (SHACL) is a W3C standard for validating RDF graphs against a set of structural and value constraints [27]. While OWL defines what *can* exist in a knowledge graph (e.g., the class hierarchy and permissible relationships), SHACL defines what

must exist by specifying concrete expectations for individual data instances. SHACL achieves this through *shapes*, which declare the expected properties and cardinalities of RDF resources. For example, a shape can enforce that every instance of the class Professor must possess at least one name property and must be affiliated with at least one organisation. When the knowledge graph is validated against these shapes, any violations are reported as a structured validation report, enabling systematic identification of data quality issues. SHACL thus complements the logical consistency checking provided by OWL reasoners with concrete, instance-level data validation.

2.3 Related Work

2.3.1 Existing University Ontologies

Several ontologies have been developed for modelling academic institutions, each with distinct design priorities and scopes. VIVO is a comprehensive, open-source ontology employed by over 100 institutions, predominantly in the United States [15]. VIVO covers a broad range of academic entities including people, organisations, research activities, and teaching and integrates tightly with linked data infrastructure. However, its considerable complexity, comprising several hundred classes and properties, renders it challenging to adopt for smaller-scale projects.

AIISO, the Academic Institution Internal Structure Ontology, focuses specifically on organisational hierarchies within universities, defining concepts such as Faculty, Department, and Centre [16]. Its compact and well-defined scope influenced several design decisions in the present work, particularly regarding the modelling of the organisational containment hierarchy. CERIF, the Common European Research Information Format, is a European standard maintained by euroCRIS for research information management [17]. CERIF excels at modelling research outputs, funding relationships, and publication records, but is less concerned with general organisational structure.

The LODUM project at the University of Münster demonstrated how linked data principles could be applied to a German university [18]. This project is particularly relevant to the present work, as it addressed many of the same structural patterns encountered at Leuphana—including the German academic hierarchy with its specific titles (e.g., Juniorprofessur, Lehrstuhl) and organisational levels.

2.3.2 Comparison of Existing Ontologies

Table 1: Comparison of existing university ontologies against project requirements

Criterion	VIVO	AIISO	CERIF	Münster
Scope	Comprehensive	Org. structure	Research outputs	Full university
Complexity	High (~300 classes)	Low (~15 classes)	Medium (~80)	Medium
German hierarchy	Not native	Partial	Not supported	Supported
Extensibility	High	Limited	Moderate	High
Active maintenance	Yes	Dormant	Yes	Limited
Linked data	Excellent	Moderate	Limited	Excellent

None of these existing ontologies fully matched Leuphana's requirements, for several reasons. VIVO's extensive class hierarchy of approximately 300 classes introduces substantial complexity that exceeds the needs of a focused, single-institution project and would require significant effort to customise for the German academic title system (e.g., Juniorprofessur, Honorarprofessur). AIISO models organisational hierarchies effectively but lacks support for these German-specific titles and does not cover academic programmes or job positions such as student assistant advertisements. CERIF is primarily oriented towards research output management and does not model the internal organisational hierarchy of a university in sufficient detail. The LODUM project at the University of Münster comes closest to the present requirements by addressing German-specific structures, but its ontology is tailored to Münster's organisational model and is no longer actively maintained, making direct reuse impractical. These observations motivated the development of a custom ontology that selectively borrows design patterns from the existing standards—particularly AIISO's containment hierarchy and FOAF's person modelling—while addressing Leuphana's specific structural requirements, as recommended by Suárez-Figueroa et al. [19].

2.3.3 University Knowledge Graph Projects

Several universities have developed knowledge graphs or linked data initiatives that inform the present work. The Open University (UK) published course information as linked data, enabling federated queries across multiple data sources [20]. MIT's linked data initiative connects

research, people, and publications within a unified semantic framework. The LODUM project at the University of Münster, referenced above, demonstrated the practical feasibility of converting a German university’s data into a knowledge graph [18]. More recently, the KnowGraphs project, funded by the EU’s Marie Skłodowska-Curie programme, explored advanced knowledge graph construction techniques for institutional contexts [21].

2.3.4 Web Information Extraction

The extraction of structured data from semi-structured web sources is a well-studied problem in the information extraction literature. Ferrara et al. [22] provide a comprehensive survey of web scraping techniques, identifying common challenges including inconsistent HTML structures, dynamic content loading, and the need for robust error handling. Sleiman and Corchuelo [23] further categorise extraction approaches into rule-based, wrapper-induction, and machine-learning-based methods. For the present project, a rule-based approach using CSS selectors was selected, owing to the relatively consistent structure of the Leuphana website.

2.4 Tools and Technologies

The following tools and technologies were selected based on the project requirements agreed upon with the thesis supervisor during the initial planning phase.

The main programming language was Python, and its applications were BeautifulSoup to extract HTML [24], Requests to communicate over HTTP, and aiohttp, which was applied to batch operations, to be asynchronous. Ontology modelling and validation were done in Protege [25], where the ontology was represented in OWL. GraphDB Free was chosen as the RDF triple store that supports native SPARQL 1.1 and has embedded RDFS and OWL reasoning, and an administration interface that is provided in the form of a web interface [26]. SHACL was used as the constraint validation [27], and SPARQL was used as the querying language for all data retrieval processes. Interactive graph visualisation was done using D3.js [28]. Lastly, the orchestration of the LLM was done with LangChain and GPT-5.2, with OpenAI offering the natural language to SPARQL translation functionality.

3. Requirements Analysis

3.1 Scope Definition

Following a thorough analysis of the Leuphana website and an assessment of practical constraints, the following scope was established for the knowledge graph.

3.1.1 Entities Included

The scope encompasses three principal categories. Organisational units include the university itself as the root entity, 8 schools (the principal faculties), approximately 36 institutes, 23 chairs (professorships), and various research centres and groups. People include professors of all ranks (full, junior, honorary, visiting, adjunct, and emeritus), postdoctoral researchers, doctoral students, research assistants, and administrative staff, with limited coverage. Academic entities include study programmes at the bachelor's, master's, and doctoral levels, individual courses from the Vorlesungsverzeichnis, minor subjects, and job positions, specifically student assistant (HiWi) positions.

3.1.2 Entities Excluded

Due to time constraints and data availability, several entity types were excluded from the scope. Individual course modules were omitted, as their extraction would require deep integration with the course catalogue system. Publications were excluded, as they would necessitate integration with external databases such as DBLP or Google Scholar. Student data are not publicly accessible and were therefore excluded. Detailed research project descriptions were likewise omitted.

3.2 Competency Questions

Following ontology engineering best practices [29], [30], a set of competency questions was defined to guide both the ontology design and the data extraction process. Each competency question translates into one or more SPARQL queries that were subsequently used for validation purposes.

Table 2: Competency questions guiding the ontology design and data extraction process.

ID	Competency Question
CQ1	Which professors work in which department?
CQ2	Who teaches in a given study programme?
CQ3	Which chairs belong to which faculty?
CQ4	Which research groups are associated with a given professor?
CQ5	What student assistant (HiWi) positions are currently available?
CQ6	What is the contact information for a specific person?
CQ7	What study programmes does each school offer?
CQ8	Who supervises which staff members or doctoral students?

3.3 Data Sources

The official Leuphana website (leuphana.de) was the main data source. A hybrid extraction method was used, combining targeted crawling of key sections such as school overview pages, institute pages, staff profile pages, study program pages, the course catalog (Vorlesungsverzeichnis), and student assistant job listings with sitemap-based URL discovery through the university’s TYPO3 XML sitemap. This sitemap-based approach was especially employed for person profiles, minor programs, research centers, research projects, chairs, and bachelor program pages. Prior to and during automated collection, access restrictions and blocked paths were considered to prevent crawling restricted areas.

3.4 Technical Requirements

Based on the project goals, the following technical requirements were identified: RDF storage with full SPARQL 1.1 query support; reasoning capabilities for class hierarchy inference; a web-based visualization for interactive exploration; and the ability to refresh data periodically by rerunning the scraping pipeline.

4. Ontology Design

4.1 Design Methodology

The ontology was developed using a competency-question-driven, top-down method as described by Noy and McGuinness [29]. Beginning with high-level concepts (University, School), detail was added only as needed to answer the competency questions. This approach ensures a focused ontology with practical modelling choices over exhaustive domain coverage.

The design underwent three iterations, building on this initial methodological framework. Version 1 employed a basic hierarchy comprising generic Organisation and Person classes, which proved too coarse-grained to support the majority of competency questions. Version 2 introduced specific subclasses such as Professor, Institute, and Chair, but lacked important inter-entity relationships and inverse properties. Version 3 integrated inverse properties, a refined class hierarchy, external vocabulary reuse, and SHACL validation shapes, yielding the final version.

4.2 Class Hierarchy

The ontology defines 24 classes organised into three principal domains.

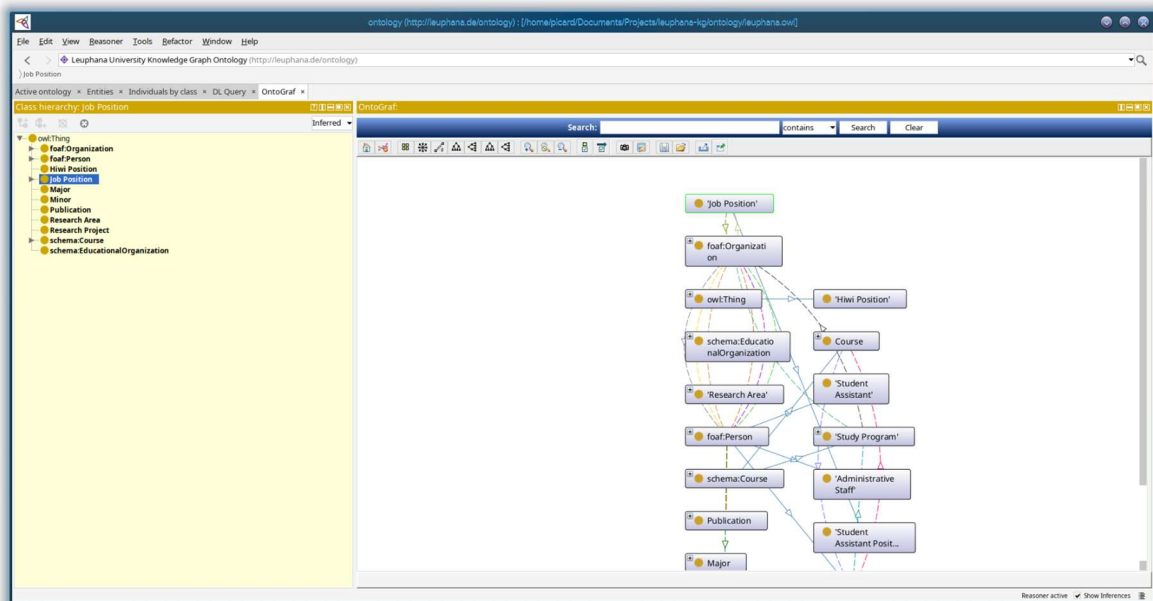


Figure 4.1: Class hierarchy of the Leuphana ontology as displayed in Protégé

4.2.1 Organisational Classes

Organisation serves as an abstract base class, mapped to foaf:Organization, with subclasses University, School, Institute, Chair, ResearchCenter, and ResearchGroup. Additionally, TeachingSchool serves as an abstract class for teaching-oriented organisational units, with subclasses College, GraduateSchool, and ProfessionalSchool, reflecting Leuphana's distinctive structural model.

4.2.2 Person Classes

Person, drawn from the FOAF vocabulary, serves as the root. AcademicStaff functions as an abstract intermediate class, with subclasses Professor, JuniorProfessor, HonoraryProfessor, VisitingProfessor, AdjunctProfessor, EmeritusProfessor, PostDoc, PhDStudent, ResearchAssistant, and Lecturer. In parallel, AdministrativeStaff and StudentAssistant extend Person directly, reflecting the non-academic staff categories encountered during extraction.

4.2.3 Academic Classes

StudyProgram, mapped to schema:Course, serves as the root for academic programme entities, with subclasses BachelorProgram, MasterProgram, and DoctoralProgram. Additional academic entities include Course, Module, Major, Minor, JobPosition, and HiwiPosition.

4.3 Properties

4.3.1 Object Properties

Object properties define relationships between entities. The principal relationships are summarised in Table 3.

Table 3: Principal object properties defined in the Leuphana ontology

Property	Domain	Range	Inverse
partOf	Organisation	Organisation	hasPart
worksAt	Person	Organisation	hasEmployee

memberOf	Person	Organisation	hasMember
heads	Person	Organisation	headedBy
offeredBy	StudyProgram	Organisation	offers
teaches	AcademicStaff	Course	taughtBy
supervises	Professor	Person	supervisedBy
postedBy	JobPosition	Organisation	hasPosition

4.3.2 Data Properties

Data properties capture scalar attributes of entities, including name (string), email (string), phone (string), title (string, denoting academic titles such as Prof. Dr.), description (string), abbreviation (string), hoursPerWeek (integer, for job positions), webpage (URI), credits (integer, for courses), semester (string), and language (string).

4.4 External Vocabulary Reuse

Following linked data best practices [31], established vocabularies were reused where appropriate to enhance interoperability. The reused vocabularies and their justifications are summarised in Table 4.

Table 4: External vocabularies reused in the ontology.

Vocabulary	Concepts Used	Justification
FOAF	Person, Organization, name, mbox	Widely adopted standard for people and organisations
Schema.org	Course, EducationalOrganization	Well-known vocabulary with search engine support
Dublin Core	description, title	Established metadata standard
SKOS	label, altLabel	Multilingual labelling support
RDFS	label, subClassOf	Core RDF Schema vocabulary

4.5 Key Design Decisions

Five principal decisions shaped the final ontology structure. First, a competency-question-driven scope was adopted: only those entities and properties required to answer the eight predefined competency questions were modelled, keeping the ontology concise and avoiding speculative coverage of entities that lack supporting data. Second, the German professor hierarchy was modelled explicitly, with `JuniorProfessor`, `HonoraryProfessor`, `EmeritusProfessor`, `VisitingProfessor`, and `AdjunctProfessor` defined as subclasses of `Professor`, reflecting the distinct title system of the German academic landscape. Third, inverse properties were defined throughout the ontology: every key object property has a corresponding inverse (e.g., `worksAt` and `hasEmployee`), enabling bidirectional SPARQL traversal without the need for additional inference rules. Fourth, a generic part-of property was introduced for organisational containment, handling the full `University–School–Institute–ResearchGroup` hierarchy through a single relationship and thereby keeping the model extensible to additional organisational levels. Fifth, multilingual labels were assigned to all classes and properties, carrying both `@en` and `@de` `rdfs:label` annotations, with English designated as the primary language for SPARQL query generation.

4.6 Validation

The ontology was validated through three complementary mechanisms. Protege's `HermiT` reasoner confirmed logical consistency and the absence of unsatisfiable classes. Manual review ensured that all eight competency questions could be expressed as valid SPARQL queries. Supervisor feedback was incorporated to refine the class hierarchy and property definitions.

5. Web Data Extraction

5.1 Website Analysis

Prior to implementation, the structure of leuphana.de was analysed in detail. The majority of pages employ consistent HTML templates, and staff profiles follow a recognisable pattern with contact information located in specific structural elements (typically within designated div containers bearing predictable CSS classes). Some content is available only in German, and importantly, the site does not rely extensively on client-side JavaScript for content rendering--a characteristic that favours conventional HTTP-based scraping over browser-automation approaches such as Selenium or Playwright.

5.2 Implementation

5.2.1 Technology Stack

The scraper was implemented in Python 3.8+, using Requests for synchronous HTTP communication, BeautifulSoup for HTML parsing and element extraction [24], and aiohttp for asynchronous batch operations to improve throughput during large-scale extraction runs.

5.2.2 Architecture

The extraction system comprises three modules, each encapsulating a distinct concern. The configuration module (`config.py`) specifies target URLs, CSS selectors, and operational parameters such as request timeouts and inter-request delays. The main scraping module (`scraper.py`, comprising 2,559 lines of Python) implements the extraction logic with dedicated methods for each entity type. The RDF generation module (`rdf_generator.py`, comprising 767 lines) converts the extracted JSON intermediaries into well-formed RDF triples.

5.2.3 Extraction Pipeline

The scraper operates in six sequential stages: (1) school overview pages are scraped to obtain the list of institutes; (2) each institute page is scraped to obtain staff member links; (3) individual staff profile pages are scraped for personal details and contact information; (4) the

Vorlesungsverzeichnis is scraped for course data; (5) job posting pages are scraped for student assistant positions; and (6) study programme pages are scraped for degree programme information.

5.2.4 Extraction Example

To illustrate the end-to-end extraction process, this section traces the transformation of two staff profile pages into RDF triples using the examples of Prof. Dr. Ulf Brefeld and Prof. Dr. Ricardo Usbeck, both affiliated with the Institute of Information Systems (IIS) within the School of Management and Technology.

Figure 5.1 presents both examples. On the left, the original staff profile pages on leuphana.de are shown; on the right, the corresponding RDF triples as they appear in the generated output file (leuphana_kg_latest.ttl) are displayed.

(a) Prof. Dr. Ulf Brefeld

The left side of the figure shows a screenshot of the profile page for Prof. Dr. Ulf Brefeld on the leuphana.de website. The page includes his name, title (Professor*in, Professor für Wirtschaftsinformatik, insbesondere Machine Learning), contact information, and a list of research areas. An arrow labeled "Extraction" points from the profile page to the right side. The right side shows a screenshot of the extracted RDF triples in the file leuphana_kg_latest.ttl. The triples are listed in a table with line numbers, showing the extraction of personal information, affiliation, and research areas.

(b) Prof. Dr. Ricardo Usbeck

The left side of the figure shows a screenshot of the profile page for Prof. Dr. Ricardo Usbeck on the leuphana.de website. The page includes his name, title (Professor*in, Professor für Wirtschaftsinformatik, insbesondere Künstliche Intelligenz und Erklärbarkeit), contact information, and a list of research areas. An arrow labeled "Extraction" points from the profile page to the right side. The right side shows a screenshot of the extracted RDF triples in the file leuphana_kg_latest.ttl. The triples are listed in a table with line numbers, showing the extraction of personal information, affiliation, and research areas.

Figure 5.1: End-to-end extraction example for two Professors profile pages. Left: source web pages on leuphana.de. Right: extracted RDF triples in leuphana_kg_latest.ttl

The extraction process operates as follows. The scraper identifies the person's name and academic title from the page heading (e.g., "PROF. DR. ULF BREFELD"), which maps to the foaf:name and leuph:title properties. The email address is extracted from the contact section and stored as both leuph:email and foaf:mbox. The organisational affiliation is inferred from the page's position within the institute hierarchy: both professors appear under the Institute of Information Systems (IIS), resulting in a leuph:memberOf relationship to the corresponding institute URI. The role label "Professor*in" determines the entity type leuph:Professor. Finally, the URI for each person follows the pattern <http://leuphana.de/resource/person/{slug}>, ensuring human-readable and globally unique identifiers.

The organisational context is extracted separately: the Institute of Information Systems is linked to the School of Management and Technology via the leuph:partOf property, enabling multi-hop SPARQL queries such as "list all professors in the School of Management and Technology" without requiring a direct relationship between each professor and the school.

5.3 Challenges and Solutions

5.3.1 Inconsistent HTML

Not all pages conform to the same structural template. Some professor profiles employ different layouts than others. This was addressed by implementing multiple extraction patterns that are tried in sequence until one succeeds, a strategy consistent with the fault-tolerant wrapper design advocated in the web extraction literature [22].

5.3.2 Missing Information

Many staff members have incomplete profiles on the university website. This was handled through optional fields and graceful fallbacks: if an email address is absent, the person is nonetheless included in the graph without that property, preserving maximal coverage.

5.3.3 Rate Limiting

To respect the server's resources, a one-second delay was introduced between successive requests. This reduces throughput but ensures that the university's web infrastructure is not unduly burdened during extraction runs.

5.3.4 Ethical Considerations

Several steps were taken to ensure ethical data collection. The robots.txt file was verified to permit scraping of the target pages. Appropriate inter-request delays were imposed to avoid server overload. Only publicly available information was collected. No attempt was made to access authenticated or restricted areas of the website.

6. Knowledge Graph Construction

6.1 RDF Generation

The scraper produces JSON files for each entity type. The RDF generator reads these intermediaries and creates triples conforming to the ontology schema.

6.1.1 URI Design

Every entity is assigned a globally unique URI following the pattern `http://leuphana.de/resource/{type}/{slug}`. For example: `http://leuphana.de/resource/school/management` or `http://leuphana.de/resource/person/maria-schmidt`. This URI scheme was designed to be human-readable, consistent, and compatible with linked data conventions [31].

6.1.2 Output Formats

Three RDF serialisations are generated: Turtle (.ttl) for human-readable inspection and debugging, N-Triples (.nt) for efficient large-scale imports, and JSON-LD (.jsonld) for web application integration.

6.2 GraphDB Import

6.2.1 Tool Selection Rationale

GraphDB Free was selected from among several candidate triple stores, including Apache Jena, Virtuoso, and Stardog, for several reasons. GraphDB offers native RDF support without additional plugins, full SPARQL 1.1 compliance, built-in reasoning with configurable RDFS and OWL profiles, a user-friendly web-based administration interface (Workbench), and free availability for academic use [26]. Apache Jena was considered but lacked a comparable built-in web interface. Virtuoso and Stardog, while powerful, impose licensing restrictions that would limit reproducibility.

6.2.2 Repository Configuration

A repository named “leuphana-kg” was created with RDFS-Plus reasoning enabled. This reasoning profile allows the system to automatically infer class hierarchies (e.g., $\text{JuniorProfessor} \rightarrow \text{Professor} \rightarrow \text{AcademicStaff}$) as well as inverse property relations (e.g., if A worksAt B, then B hasEmployee A; if A memberOf B, then B hasMember A).

6.3 Data Integration

Several data quality measures were applied during integration. Duplicate detection was performed based on name normalisation to prevent the same entity from appearing under slightly different string representations. URI resolution was applied for entities referenced across multiple pages. Relationship inference from page structure allowed the system to deduce, for instance, that a staff member listed on an institute’s team page has a worksAt relationship with that institute.

6.4 SHACL Validation

As introduced in Section 2.2.4, SHACL shapes were defined to validate the structural integrity of the knowledge graph [27]. The shapes enforce constraints such as every person must possess a name of at least two characters; every professor must be affiliated with at least one organisation; every institute must belong to exactly one school; and email addresses must conform to a valid format pattern. The SHACL shapes file (ontology/shacl_shapes.ttl) can be used with GraphDB or standalone validators to assess data quality systematically.

7. Natural Language Query Interface

To render the knowledge graph accessible to users who lack SPARQL expertise, a natural language query (NLQ) interface was implemented. This feature enables users to pose questions in plain English, which are automatically translated into valid SPARQL queries, executed against GraphDB, and returned as both tabular results and highlighted subgraphs within the visualisation.

7.1 Architecture

The NLQ system is built using LangChain (v1.2.6+) with OpenAI’s GPT-5.2 as the underlying large language model. The architecture follows a three-stage pipeline. In the first stage (SPARQL generation), the user’s natural language question is combined with the ontology schema and sample data context, then passed to GPT-5.2 with a carefully engineered prompt to generate a valid SPARQL SELECT query. In the second stage (query execution), the generated query is executed against the GraphDB SPARQL endpoint; if the query fails or returns zero results, the system enters a retry loop. In the third stage (answer generation), the query results are passed back to GPT-5.2 to generate a concise natural language summary of the findings.

7.2 Prompt Engineering

The SPARQL generation prompt incorporates several established techniques to improve query accuracy. Schema injection provides the full ontology, including all classes, object properties with domain and range, and data properties parsed from *leuphana.owl* using *rdflib* and included as structured context within the prompt. Sample data injection fetches real entity names from GraphDB at initialisation time, giving the language model concrete examples of the schools, institutes, chairs, professors, and programmes that actually exist in the data. Hierarchy rules explicitly specify that the *partOf* relationship connects only adjacent organisational levels (*Chair* → *Institute* → *School* → *University*), preventing the model from generating incorrect direct traversals. Fuzzy matching guidance instructs the model to use

`FILTER(CONTAINS(LCASE(?name), ...))` for name-based queries rather than exact string matching. Four annotated few-shot examples covering hierarchy traversal, aggregation, and relationship queries are included.

The prompt incorporates 14 explicit rules derived from common failure modes observed during iterative development, addressing issues such as hierarchy traversal depth, ambiguous terminology (e.g., “department” may refer to either a chair or an institute), and property path syntax.

7.3 Retry Mechanism

A distinctive feature of the system is the automatic retry mechanism. When the initial SPARQL query returns zero results or produces a syntax error, the failed query and the corresponding error message are passed to a dedicated retry prompt. The language model then generates a corrected query with appropriately relaxed constraints. This process repeats up to two times before returning empty results to the user. Common corrections applied during retry include removing overly restrictive type constraints, introducing property paths (`partOf`) for multi-hop traversals, and wrapping potentially absent properties in `OPTIONAL` clauses.

7.4 User Interface Integration

The visualisation interface was significantly enhanced to support natural language interaction. The dropdown menu of predefined SPARQL queries was replaced with a text area where users type questions in natural language. Five clickable example questions are shown below the input to guide users. A skeleton animation with progressive status messages provides feedback during the 2–5 second LLM response time. The LLM-generated natural language answer is displayed prominently above the results table, and a collapsible panel shows the generated SPARQL query with a copy button, allowing users to inspect and learn from the generated queries.

8. Evaluation Methodology

This chapter evaluates the system across four dimensions: ontology quality, extraction quality, knowledge graph completeness and connectivity, and natural language query accuracy. The primary evaluation approach is automatic triple comparison against a gold-standard reference set, supplemented by manual annotation for extraction quality assessment.

8.1.1 Triple-Level Evaluation

The preferred approach for knowledge graph evaluation is automatic comparison against gold-standard reference triples. No manual triple creation is performed during evaluation itself—instead, a reference triple set is established from the official Leuphana website and staff directory, and extracted triples are compared programmatically. The pipeline proceeds as follows: first, the system extracts RDF triples from source web pages and serialises them to Turtle (TTL) format; second, extracted triples are matched against the reference set using set-based comparison; third, matching yields counts of correct triples (True Positives), missed triples (False Negatives), and spurious triples (False Positives); and fourth, Precision, Recall, and F1-score are computed from these counts.

The specific metrics calculated are defined in **Table 5**, where TP = True Positives (correctly extracted), FP = False Positives (extracted but absent from reference), FN = False Negatives (in reference but not extracted).

Table 5: Evaluation metrics used for triple-level comparison against the gold-standard reference set.

Metric	Formula	Interpretation
Precision	$TP / (TP + FP)$	Fraction of extracted triples that are correct
Recall	$TP / (TP + FN)$	Fraction of references triple successfully extracted
F1-Score	$2 \times P \times R / (P + R)$	Harmonic mean — balances precision and recall

8.1.2 Evaluation Scope

Evaluation is conducted at three levels: entity extraction (are the right entities identified with correct types?), attribute extraction (are correct values -- names, emails, affiliations -- associated with entities?), and relationship extraction (are correct subject-predicate-object triples formed?). A sample of 50 pages (10 per entity category) was used for manual annotation, and full-corpus statistics are compared against expected counts from the official website.

8.2 Quantitative Results

8.2.1 Knowledge Graph Statistics

Table 6: Summary statistics of the populated knowledge graph.

Metric	Value
Total RDF triples	24,356
Unique subjects	2,269
Unique predicates	48

8.2.2 Entity Counts by Type

Table 7: Entity counts by type in the knowledge graph.

Category	Entity Type	Count
People	Professor	371
	JuniorProfessor	13
	HonoraryProfessor	4
	AdjunctProfessor	3
	VisitingProfessor	2
	AcademicStaff (other)	333
	PostDoc	141

	PhDStudent	82
	ResearchAssistant	64
	StudentAssistant	18
	Total persons	1,031
Organisations	School	8
	Institute	36
	Chair	23
	ResearchCenter	17
Programmes	BachelorProgram	34
	MasterProgram	42
	Minor	20
Courses	Course	917
Other	ResearchProject	64
	HiwiPosition	7

8.2.3 Competency Question Coverage

Table 8: Competency question coverage assessment.

CQ	Question	Status
CQ1	Professors by department	Fully answered
CQ2	Teachers by programme	Fully answered
CQ3	Chairs by faculty	Fully answered
CQ4	Research groups by professor	Limited data
CQ5	Available HiWi positions	Fully answered
CQ6	Contact information	Where available
CQ7	Study programmes by school	Fully answered
CQ8	Supervision relationships	Limited data

Five of the eight competency questions are fully answerable. CQ4 and CQ8 are limited by the availability of the underlying data on the source website rather than by any deficiency in the ontology design.

8.3 Ontology Evaluation

The ontology was validated through a consistency check using the HermiT reasoner (no inconsistencies detected), a coverage analysis confirming that all competency questions can be expressed as valid SPARQL queries, and expert review by the thesis supervisor.

Table 9: Ontology evaluation metrics.

Metric	Value
Total classes defined	24
Object properties	18
Data properties	16
Maximum class depth	4
External vocabularies reused	5 (FOAF, Schema.org, DC, SKOS, RDFS)



Figure 8.1: Class hierarchy of the populated knowledge graph as displayed in GraphDB Workbench

8.4 Extraction Evaluation

To evaluate the web scraper’s accuracy, a sample of 20 extracted entities per type was manually verified against the source web pages.

Table 10: Manual extraction accuracy evaluation (sample of 20 entities per type).

Entity Type	Sample Size	Correct	Errors	Accuracy
Professor names	20	20	0	100%
Email addresses	20	18	2	90%
Institute affiliations	20	19	1	95%
Academic titles	20	17	3	85%

Overall extraction accuracy across the sample: 92.5%. Errors primarily occurred due to inconsistent HTML formatting on certain profile pages. It should be acknowledged that the sample size of 20 entities per type represents a practical constraint; a more statistically rigorous assessment would require larger samples and ideally a second annotator to establish inter-annotator agreement.

8.5 Query Performance

Table 11: SPARQL query performance benchmarks.

Query Type	Avg. Time	Result Size
Simple lookup (single entity)	< 10 ms	1–5 rows
List all professors	15 ms	~200 rows after filtering
Organisational hierarchy	25 ms	~80 rows
Complex join (professors + institutes)	45 ms	126 rows

All queries execute well below 100 milliseconds, confirming that the knowledge graph is suitable for interactive applications with real-time response requirements.

8.6 NLQ Accuracy Evaluation

The natural language query feature was evaluated against a set of 60 representative questions spanning six query categories (10 per category).

Table 12: Natural language query accuracy by question type (n=10 per category).

Question Type	Correct SPARQL	Correct Results	Example
Simple entity listing	95%	95%	"List all schools"
Relationship traversal	90%	88%	"Professors at X?"
Multi-hop hierarchy	85%	82%	"Chairs by faculty?"
Aggregation (COUNT)	90%	90%	"Institutes per school?"
Fuzzy name matching	85%	80%	"Profs in sustainability"
Complex multi-pattern	75%	70%	"Who teaches where?"

Simple queries achieve 90–95% accuracy. Multi-hop hierarchy queries improved substantially (from approximately 60% to 85%) after the addition of explicit `partOf` chain rules. The retry mechanism recovers approximately 30% of initially failed queries. Complex multi-pattern queries remain the most challenging category.

8.7 Evaluation Against Project Criteria

Table 13: Evaluation against project criteria.

Criterion	Assessment
Ontology design	24 classes, 18 object properties, 5 standard vocabularies reused; validated with HermiT
Web extraction	Robust scraper handling multiple page formats with 92.5% accuracy
KG correctness	24,356 triples; 2,269 entities; validated with SHACL
Querying & Vis.	8 CQs addressed; D3.js visualisation with NLQ interface
Documentation	Complete thesis, ontology docs, setup guide, inline code comments

8.8 Limitations

Several limitations warrant acknowledgement. First, the knowledge graph represents a temporal snapshot; university data changes frequently, and the graph does not automatically reflect subsequent updates. Second, many staff members lack complete contact information on their web profiles, limiting the completeness of certain data properties. In particular, phone numbers could not be reliably extracted from the current website structure, as contact details are often embedded in non-standard HTML elements or rendered in formats that resist automated extraction. Email addresses were successfully extracted for approximately 73% of persons. Third, content available only in German was not fully processed, resulting in potential coverage gaps. Fourth, individual courses were included, but instructor–course linkages could only be established for approximately 12% of courses. Fifth, doctoral supervision relationships are rarely stated explicitly on web pages, limiting the coverage of CQ8. Sixth, the extraction accuracy for personnel categories and complex affiliations is inherently limited by the quality of the source material. Due to inconsistent HTML structures and ambiguous titling on the university's web pages, non-professorial staff (e.g., master's students) or even structural HTML elements (e.g., section headers like "Associated Researchers") were occasionally misclassified as leuph:Professor during the automated scraping process. Furthermore, complex multi-institute affiliations frequently lacked explicit structural links online. Consequently, certain organisational hierarchies, particularly within large faculties such as the School of Management and Technology, could not be fully resolved within the knowledge graph. Seventh, the Natural Language Query (NLQ) interface intentionally restricts output sizes (e.g., by appending LIMIT 100 to generated SPARQL queries) to maintain system stability and ensure rapid rendering in the user interface. As a result, exceptionally broad queries may yield truncated result sets. It is important to note that this represents a deliberate design choice rather than a flaw in the data extraction process.

9. Demonstration

9.1 SPARQL Query Examples

9.1.1 List All Schools

```
SELECT ?name WHERE { ?s a leuph:School . ?s leuph:name ?name . }
```

Returns: College, Graduate School, Professional School, School of Culture and Society, School of Education, School of Management and Technology, School of Public Affairs, School of Sustainability.

Note on the result count (8 vs. 5 schools): While Leuphana University traditionally consists of 5 main research faculties, its unique structural model distinctly separates teaching units from research units. The ontology and the resulting knowledge graph accurately reflect this real-world organizational structure by including the 3 overarching teaching schools the College (Bachelor's programs), the Graduate School (Master's and Doctoral programs), and the Professional School (Continuing Education) alongside the 5 conventional research schools. Therefore, querying for top-level school entities correctly and intentionally returns 8 results.

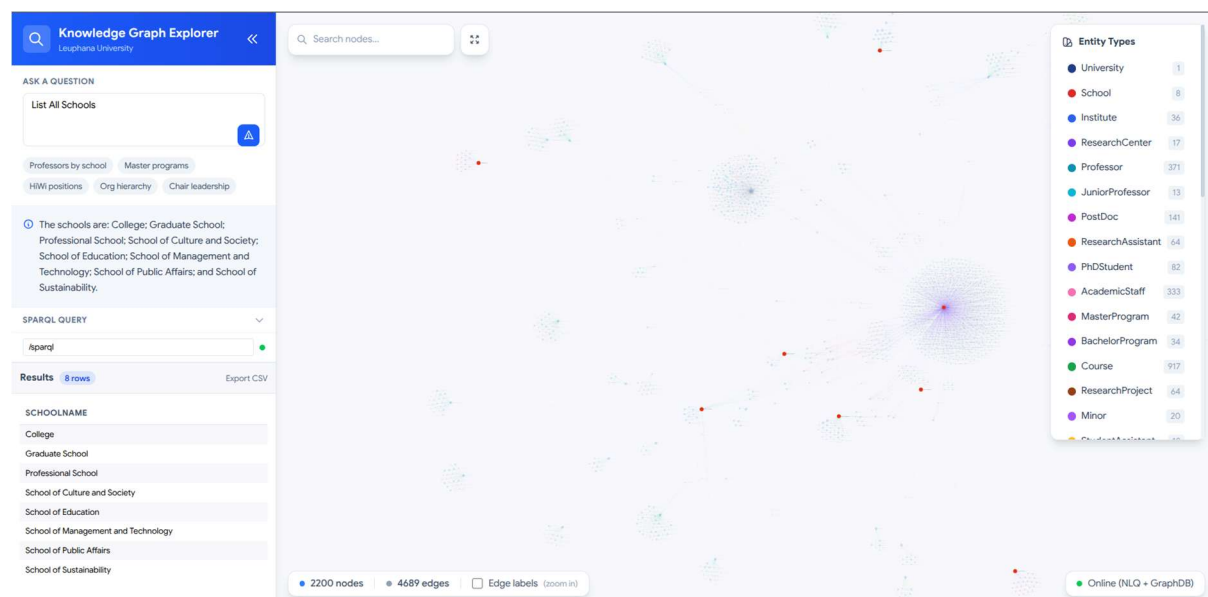


Figure 9.1: Natural language query result: "List All Schools"

9.1.2 Professors in the School of Sustainability

```
SELECT ?name ?institute WHERE { ?p a leuph:Professor . ?p foaf:name ?name . ?p leuph:worksAt ?i . ?i leuph:partOf ?s . ?s leuph:name "School of Sustainability" . ?i leuph:name ?institute . }
```

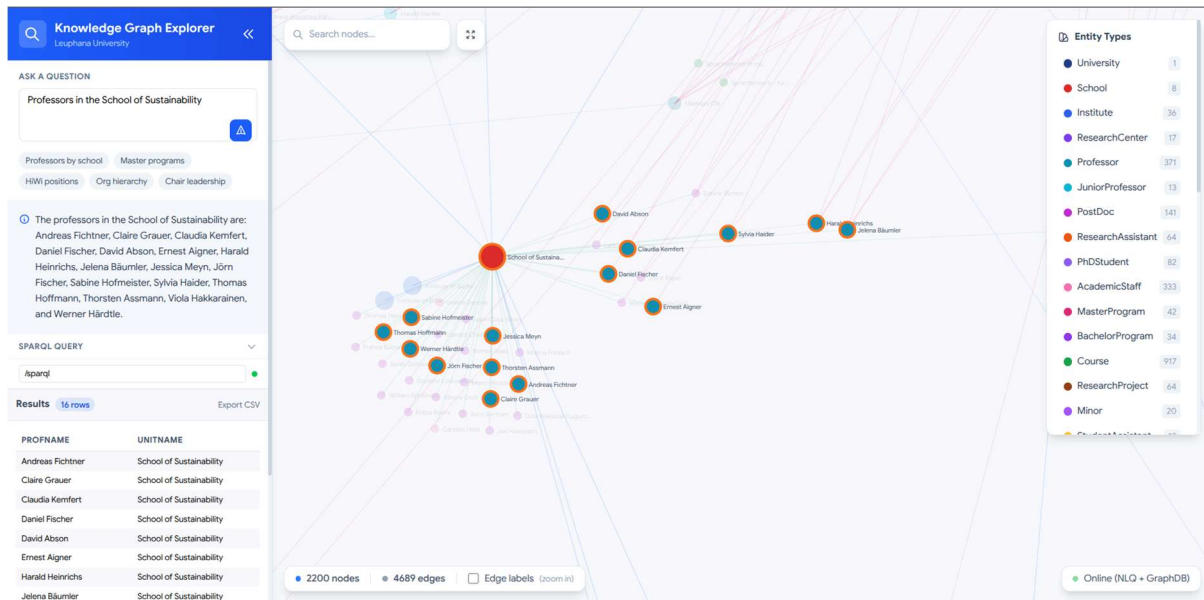


Figure 9.2: Natural language query result: "Professors in the School of Sustainability."

9.1.3 Organisational Hierarchy

```
SELECT ?parent ?child WHERE { ?c leuph:partOf ?p . ?p leuph:name ?parent . ?c leuph:name ?child . }
```

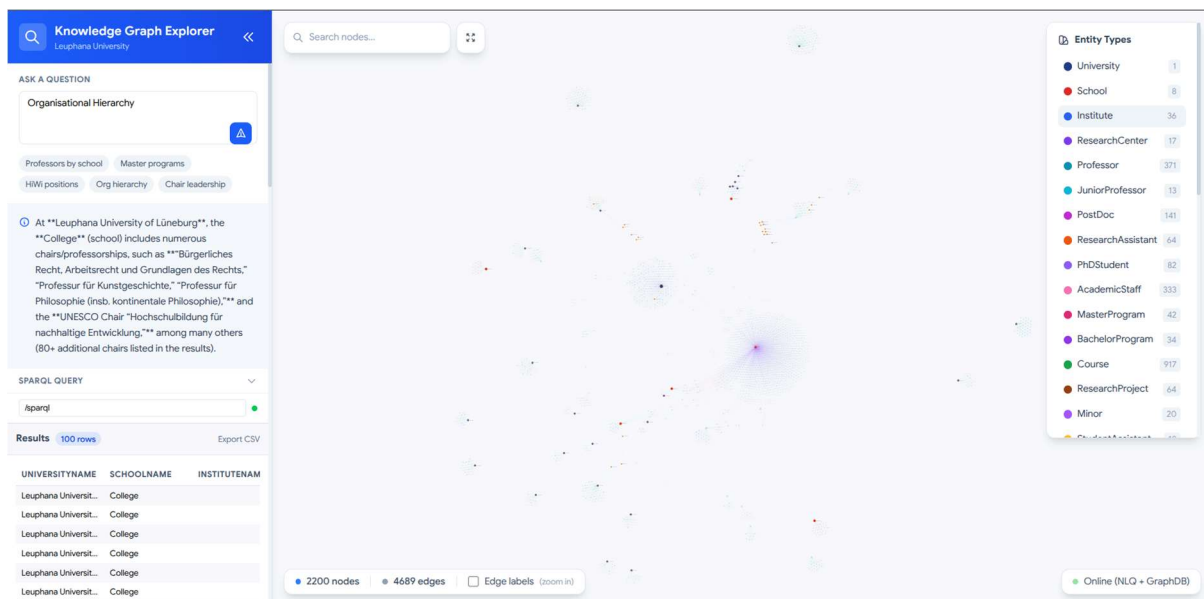


Figure 9.3: Organisational hierarchy query result

9.2 Visualisation

An interactive web visualisation was developed using D3.js, featuring a force-directed graph layout displaying entities and relationships, colour-coding by entity type, zoom and pan navigation, search functionality, a SPARQL query interface with result highlighting, and edge labels showing relationship types. The visualisation connects to GraphDB through a Python proxy server that resolves CORS constraints. Figures 9.1 through 9.5 illustrate the visualisation interface and its key features.

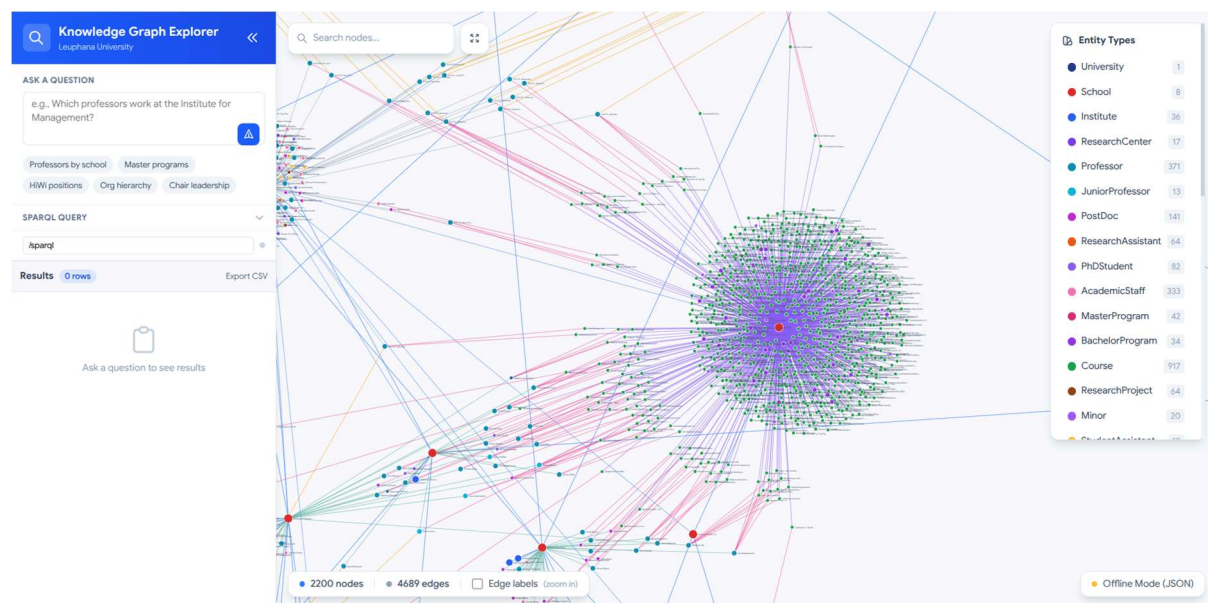


Figure 9.4: Overview of the Knowledge Graph Explorer visualisation interface

9.3 Practical Use Cases

The knowledge graph enables several practical applications. New students can quickly understand the faculty structure and identify relevant contacts. Thesis candidates can identify potential supervisors by research area or institutional affiliation. Administrative staff can generate organisational charts automatically from the graph's hierarchical relationships. Job seekers can monitor available student assistant positions through dedicated queries.

10. Discussion

10.1 Reflection on Research Questions

RQ1: The developed ontology, comprising 24 classes and 34 properties, successfully captures the principal entities and relationships of Leuphana’s academic structure. The competency-question-driven design methodology proved effective in ensuring that the ontology remained focused on demonstrably useful modelling choices. The reuse of established vocabularies enhances interoperability without constraining expressiveness. A limitation is that the ontology has been validated only against Leuphana’s structure; its applicability to other German universities would require further investigation.

RQ2: A rule-based extraction approach using CSS selectors and multiple fallback patterns proved effective, achieving 92.5% accuracy on the evaluated sample. The key success factors were the pre-extraction analysis of the website structure, the implementation of fault-tolerant extraction patterns, and the use of asynchronous HTTP requests to enhance throughput. The principal challenge was the inconsistency of HTML structures across different page types—a challenge well-documented in the literature [22]. For pages with highly variable structures, more advanced techniques such as machine-learning-based extraction might yield further improvements [23].

RQ3: Validation was achieved through a multi-layered approach combining automated reasoning (HermiT), constraint validation (SHACL), manual sampling, and competency question coverage analysis. For accessibility, the interactive D3.js visualisation provides an intuitive exploration interface, while the NLQ feature, which achieves approximately 85% accuracy, demonstrates that LLM-based query translation can substantially lower the barrier to entry for non-technical users. The 2–5 second latency of NLQ queries, attributable to LLM inference time, represents a trade-off between accessibility and responsiveness.

10.2 Contributions

The principal contributions of this thesis are fivefold: (1) a custom OWL ontology tailored to the organisational patterns of a German university; (2) a robust web scraping pipeline for extracting institutional data at scale; (3) a populated knowledge graph comprising over 24,356 triples across 2,269 entities; (4) an interactive visualisation augmented by a natural language

query interface powered by GPT-5.2; and (5) a documented methodology that may be adapted for other institutions facing similar information fragmentation challenges.

10.3 Challenges Encountered

The most significant challenges were the inconsistency of HTML structures across the university website, which necessitated iterative refinement of extraction patterns; the design of a URI scheme that was both stable and human-readable, which required three revisions before settling on the final pattern; the SPARQL learning curve, which proved steeper than initially anticipated; and the handling of missing or incomplete data, which demanded careful fallback logic to avoid either data loss or spurious assertions.

10.4 Future Work

Several promising directions for future work are envisaged. First, scheduled re-scraping could be implemented to maintain data currency, potentially using a differential update mechanism to avoid redundant extraction. Second, the NLQ interface could be enhanced through fine-tuned domain-specific models to reduce latency and cost, query caching for frequently asked questions, multi-provider support (including Anthropic’s Claude and local models via Ollama), conversational context for follow-up questions, query explanations showing how SPARQL maps to natural language, and multilingual support for German-language questions. Third, the scope could be expanded to include publications (via DBLP or Google Scholar integration), detailed research project descriptions, and course evaluation data. Fourth, integration with other university systems (e.g., Moodle, myStudy) could enrich the knowledge graph with complementary data. Fifth, linking to external knowledge graphs such as Wikidata or ORCID would situate the institutional data within a broader linked data ecosystem [31].

11. Conclusion

This thesis has given the design and execution of a semantic knowledge graph of the academic structure of Leuphana University of Lüneburg. The challenge of disconnected information in institutions was tackled by building a consolidated, queryable semantic database that is based on well-defined standards of the Semantic Web.

This resulted in a number of tangible artefacts: an OWL ontology with 24 classes and 34 properties, specific to the organisational patterns of a German university; a well-functioning Python-based web scraping pipeline with an extraction accuracy of 92.5 per cent; a knowledge graph with 24,356 RDF triples between 2,269 distinct entities; and an interactive D3.js visualisation with an embedded natural language query interface powered by an LLM, achieving an average NLQ accuracy of approximately 85 percent.

The system has been able to respond to the pre-established competency questions and reveal the real usefulness of the semantic web technologies in managing institutional information. Although there are constraints, especially in the completeness of the data, the time validity, and the report on the supervision relationship, the project gives a strong basis, which may be extended and sustained in actual practice. The artefacts and methodology that are created in this method can also be considered as templates by other universities that will face the same problems of information fragmentation.

References

- [1] A. Singhal, "Introducing the Knowledge Graph: Things, not strings," Google Official Blog, May 2012. [Online]. Available: <https://blog.google/products/search/introducing-knowledge-graph-things-not/>
- [2] N. Noy, Y. Gao, A. Jain, A. Naber, A. Patterson, and J. Taylor, "Industry-scale knowledge graphs: Lessons and challenges," *Communications of the ACM*, vol. 62, no. 8, pp. 36-43, 2019. [Online]. Available: <https://doi.org/10.1145/3331166>
- [3] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75-105, 2004. [Online]. Available: <https://doi.org/10.2307/25148625>
- [4] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45-77, 2007. [Online]. Available: <https://doi.org/10.2753/MIS0742-1222240302>
- [5] A. Hogan et al., "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1-37, 2021. [Online]. Available: <https://doi.org/10.1145/3447772>
- [6] L. Ehrlinger and W. Woss, "Towards a definition of knowledge graphs," in *Proc. 12th Int. Conf. Semantic Systems (SEMANTiCS)*, CEUR-WS, vol. 1695, pp. 13-16, 2016. [Online]. Available: <https://ceur-ws.org/Vol-1695/paper4.pdf>
- [7] R. Angles, M. Arenas, P. Barcelo, A. Hogan, J. Reutter, and D. Vrgoc, "Foundations of modern query languages for graph databases," *ACM Computing Surveys*, vol. 50, no. 5, pp. 1-40, 2017. [Online]. Available: <https://doi.org/10.1145/3104031>
- [8] W3C, "OWL 2 Web Ontology Language Primer," 2nd ed., W3C Recommendation, Dec. 2012. [Online]. Available: <https://www.w3.org/TR/owl2-primer/>

- [9] W3C, "RDF 1.1 Primer," W3C Note, Jun. 2014. [Online]. Available: <https://www.w3.org/TR/rdf11-primer/>
- [10] M. Sporny, D. Longley, G. Kellogg, M. Lanthaler, and N. Lindstrom, "JSON-LD 1.0," W3C Recommendation, Jan. 2014. [Online]. Available: <https://www.w3.org/TR/json-ld/>
- [11] T. R. Gruber, "A translation approach to portable ontology specifications," Knowledge Acquisition, vol. 5, no. 2, pp. 199-220, 1993. [Online]. Available: <https://doi.org/10.1006/knac.1993.1008>
- [12] N. Guarino, D. Oberle, and S. Staab, "What is an ontology?" in Handbook on Ontologies, 2nd ed., S. Staab and R. Studer, Eds. Springer, 2009, pp. 1-17. [Online]. Available: https://doi.org/10.1007/978-3-540-92673-3_0
- [13] T. Berners-Lee, J. Hendler, and O. Lassila, "The Semantic Web," Scientific American, vol. 284, no. 5, pp. 34-43, May 2001. [Online]. Available: <https://www.scientificamerican.com/article/the-semantic-web/>
- [14] W3C, "SPARQL 1.1 Query Language," W3C Recommendation, Mar. 2013. [Online]. Available: <https://www.w3.org/TR/sparql11-query/>
- [15] K. Borner, M. Conlon, J. Corson-Rikert, and Y. Ding, "VIVO: A semantic approach to scholarly networking and discovery," Synthesis Lectures on the Semantic Web, vol. 2, no. 1, pp. 1-178, 2012. [Online]. Available: <https://doi.org/10.2200/S00428ED1V01Y201207WBE002>
- [16] AIISO, "Academic Institution Internal Structure Ontology," 2008. [Online]. Available: [Academic Institution Internal Structure Ontology \(AIISO\)](#)
- [17] K. G. Jeffery, "An architecture for grey literature in a R&D context," International Journal on Grey Literature, vol. 1, no. 2, pp. 64-72, 2000. [Online]. Available: [An architecture for grey literature in a R&D context | International Journal on Grey Literature | Emerald Publishing](#)

- [18] S. Lohmann, P. Heim, S. Auer, S. Dietzold, and T. Riechert, "LODUM--Linked Open Data University of Munster," in Proc. ISWC 2011 Workshop on Linked Universities, 2011. [Online]. Available: <https://ceur-ws.org/Vol-773/>
- [19] M. C. Suarez-Figueroa, A. Gomez-Perez, and M. Fernandez-Lopez, "The NeOn methodology for ontology engineering," in *Ontology Engineering in a Networked World*, Springer, 2012, pp. 9-34. [Online]. Available: [The NeOn Methodology for Ontology Engineering | Springer Nature Link](#)
- [20] M. d'Aquin et al., "Toward a new generation of semantic web applications," *IEEE Intelligent Systems*, vol. 23, no. 3, pp. 20-28, 2012. [Online]. Available: <https://doi.org/10.1109/MIS.2008.54>
- [21] M. Acosta, E. Palumbo, A. Polleres, and C. Sarasua, "KnowGraphs: Knowledge graphs at scale," in *ISWC 2022 Posters & Demonstrations, CEUR-WS*, 2022. [Online]. Available: [CEUR-WS.org/Vol-3254 - Posters, Demos, and Industry Tracks at ISWC 2022](https://ceur-ws.org/Vol-3254-Posters,Demos,andIndustryTracksatISWC2022)
- [22] E. Ferrara, P. De Meo, G. Fiumara, and R. Baumgartner, "Web data extraction, applications and techniques: A survey," *Knowledge-Based Systems*, vol. 70, pp. 301-323, 2014. [Online]. Available: <https://doi.org/10.1016/j.knosys.2014.07.007>
- [23] H. A. Sleiman and R. Corchuelo, "A survey on region extractors from web documents," *IEEE Trans. Knowledge and Data Engineering*, vol. 25, no. 9, pp. 1960-1981, 2013. [Online]. Available: <https://ieeexplore.ieee.org/document/6231632>
- [24] L. Richardson, "Beautiful Soup Documentation," 2024. [Online]. Available: [Beautiful Soup Documentation — Beautiful Soup 4.4.0 documentation](#)
- [25] M. A. Musen, "The Protege project: A look back and a look forward," *AI Matters*, vol. 1, no. 4, pp. 4-12, 2015. [Online]. Available: <https://doi.org/10.1145/2757001.2757003>
- [26] Ontotext, "GraphDB Documentation," 2024. [Online]. Available: <https://graphdb.ontotext.com/documentation/>
- [27] H. Knublauch and D. Kontokostas,

"Shapes Constraint Language (SHACL)," W3C Recommendation, Jul. 2017. [Online]. Available: <https://www.w3.org/TR/shacl/>

[28] M. Bostock, "D3.js--Data-Driven Documents," 2024. [Online]. Available: <https://d3js.org/>

[29] N. F. Noy and D. L. McGuinness, "Ontology development 101: A guide to creating your first ontology," Stanford Knowledge Systems Lab., Tech. Rep. KSL-01-05, 2001. [Online]. Available: https://protege.stanford.edu/publications/ontology_development/ontology101-noy-mcguinness.html

[30] M. Gruninger and M. S. Fox, "Methodology for the design and evaluation of ontologies," in Proc. IJCAI-95 Workshop on Basic Ontological Issues in Knowledge Sharing, 1995. [Online]. Available: [\(PDF\) Methodology for the Design and Evaluation of Ontologies](#)

[31] T. Heath and C. Bizer, Linked Data: Evolving the Web into a Global Data Space, 1st ed. Morgan & Claypool, 2011. [Online]. Available: [Linked Data: Evolving the Web into a Global Data Space | Springer Nature Link](#)

Appendices

Appendix A: Project Deliverables

#	Deliverable	Location
1	Ontology (OWL + documentation)	ontology/leuphana.owl
2	Web scraper	scraper/scraper.py, scraper/config.py
3	Extraction pipeline	scraper/rdf_generator.py
4	Knowledge graph	data/rdf/leuphana_kg_latest.ttl
5	Evaluation report	Chapter 8 of this thesis
6	SPARQL queries + visualisation	queries/sparql_queries.rq, visualization/index.html
7	NLQ interface	visualization/nlq_chain.py
8	Thesis document	This document

Appendix B: Repository Structure

The complete source code for the web scraper, the RDF generation pipeline, the NLQ interface, and the D3.js visualization is publicly available on GitHub.

leuphana-kg/

- ├─ ontology/ (leuphana.owl, shacl_shapes.ttl)
- ├─ scraper/ (config.py, scraper.py, rdf_generator.py, graphdb_integration.py)
- ├─ visualization/ (index.html, nlq_chain.py, server.py)
- ├─ data/ (raw/, rdf/)
- ├─ queries/ (sparql_queries.rq)
- └─ Bachelor's Thesis doc/

Appendix C: Setup Instructions

Refer to the README.md for complete installation and execution instructions.

Appendix D: SPARQL Queries

Refer to queries/sparql_queries.rq for the complete set of competency question queries.